R Code

```r
# ARMAX Assignment: S&P 500 (Output) & Nasdaq (Input)
#  3: Load data and compute log returns

dat <- read.csv("/Users/manishsm/Desktop/Spring 2026/Time Series Analysis/Assignments/quantmod_dataset.csv", stringsAsFactors = FALSE)
dat$Date <- as.Date(dat$Date)
dat <- dat[order(dat$Date), ]

# Log returns of Adjusted Close
# Output (Y): S&P 500 log returns
# Input  (X): Nasdaq log returns
sp500_adj  <- dat$GSPC.Adjusted
nasdaq_adj <- dat$IXIC.Adjusted

sp500_ret  <- diff(log(sp500_adj))   # length N-1
nasdaq_ret <- diff(log(nasdaq_adj))

dates_ret  <- dat$Date[-1]

cat(" Data Summary \n")
cat("Observations (log returns):", length(sp500_ret), "\n")
cat("Date range:", format(min(dates_ret)), "to", format(max(dates_ret)), "\n\n")

# Quick plot of log returns
par(mfrow = c(2,1), mar = c(3,4,2,1))
plot(dates_ret, sp500_ret,  type = "l", col = "blue",
     main = "S&P 500 Log Returns (Output Y)",  ylab = "Log Return", xlab = "")
plot(dates_ret, nasdaq_ret, type = "l", col = "orange",
     main = "Nasdaq Log Returns (Input X)", ylab = "Log Return", xlab = "Date")


# ACF/PACF to guide AR/MA order selection
par(mfrow = c(2,2))
acf(sp500_ret,  main = "ACF  - S&P 500 Returns",  lag.max = 30)
pacf(sp500_ret, main = "PACF - S&P 500 Returns",  lag.max = 30)
acf(nasdaq_ret,  main = "ACF  - Nasdaq Returns",   lag.max = 30)
pacf(nasdaq_ret, main = "PACF - Nasdaq Returns",   lag.max = 30)
par(mfrow = c(1,1))



#  4 & 5: Fit ARMAX models and compare with AIC, BIC, Chi-square (nested)
#
# We use Arima() from {forecast} with xreg = nasdaq_ret (the exogenous input).
# Models considered:
#   M1: ARMAX(1,0,0) -- AR(1) noise
#   M2: ARMAX(0,0,1) -- MA(1) noise
#   M3: ARMAX(1,0,1) -- ARMA(1,1) noise
```

```r
#   M4: ARMAX(2,0,1) -- ARMA(2,1) noise
#   M5: ARMAX(1,0,2) -- ARMA(1,2) noise
#   M6: ARMAX(2,0,2) -- ARMA(2,2) noise (most general)

X <- matrix(nasdaq_ret, ncol = 1)   # exogenous regressor matrix

fit_armax <- function(p, q, xreg = X) {
  Arima(sp500_ret, order = c(p, 0, q), xreg = xreg, include.mean = TRUE)
}

cat(" Fitting ARMAX Models \n")
M1 <- fit_armax(1, 0)  # AR(1)
M2 <- fit_armax(0, 1)  # MA(1)
M3 <- fit_armax(1, 1)  # ARMA(1,1)
M4 <- fit_armax(2, 1)  # ARMA(2,1)
M5 <- fit_armax(1, 2)  # ARMA(1,2)
M6 <- fit_armax(2, 2)  # ARMA(2,2)

models      <- list(M1 = M1, M2 = M2, M3 = M3, M4 = M4, M5 = M5, M6 = M6)
model_names <- c("ARMAX(1,0)", "ARMAX(0,1)", "ARMAX(1,1)",
                 "ARMAX(2,1)", "ARMAX(1,2)", "ARMAX(2,2)")

# Build comparison table
aic_vals <- sapply(models, AIC)
bic_vals <- sapply(models, BIC)
loglik   <- sapply(models, logLik)
npar     <- sapply(models, function(m) length(coef(m)) + 1)  # +1 for sigma^2

comparison <- data.frame(
  Model    = model_names,
  AIC      = round(aic_vals, 2),
  BIC      = round(bic_vals, 2),
  LogLik   = round(as.numeric(loglik), 2),
  nParams  = npar
)
comparison <- comparison[order(comparison$BIC), ]
cat("\n Model Comparison (sorted by BIC) \n")
print(comparison)

# Chi-square tests for nested pairs
# M1 vs M3 (M1 nested in M3): adding MA(1)
# M2 vs M3 (M2 nested in M3): adding AR(1)
# M3 vs M4 (M3 nested in M4): adding AR(2)
# M3 vs M5 (M3 nested in M5): adding MA(2)
# M4 vs M6 (M4 nested in M6): adding MA(2)
# M5 vs M6 (M5 nested in M6): adding AR(2)

lrt <- function(small, large, label) {
  stat <- 2 * (as.numeric(logLik(large)) - as.numeric(logLik(small)))
  df   <- length(coef(large)) - length(coef(small))
  pval <- pchisq(stat, df = df, lower.tail = FALSE)
  cat(sprintf("  %-20s  chi2=%.3f  df=%d  p=%.4f\n", label, stat, df, pval))
}

```

```r
cat("\n Likelihood Ratio (Chi-square) Tests for Nested Models \n")
lrt(M1, M3, "M1 vs M3 (+MA1)")
lrt(M2, M3, "M2 vs M3 (+AR1)")
lrt(M3, M4, "M3 vs M4 (+AR2)")
lrt(M3, M5, "M3 vs M5 (+MA2)")
lrt(M4, M6, "M4 vs M6 (+MA2)")
lrt(M5, M6, "M5 vs M6 (+AR2)")

#
#  6: Choose best model and justify
#
# Criteria: lowest BIC (penalizes complexity more) + chi-square significance.
# Inspect comparison table above; typically ARMAX(1,1) or ARMAX(1,0) is best.

best_name  <- comparison$Model[1]
cat(paste0("\n Best Model (lowest BIC): ", best_name, " \n"))

# Map best_name back to fitted object
best_model <- models[[which(model_names==best_name)]]
cat("\nBest model summary:\n")
print(summary(best_model))

#
#  Residual checking white noise
#
resid_best <- residuals(best_model)

# Autocovariance / ACF of residuals
par(mfrow = c(2, 2))
acf(resid_best,  main = "ACF of Residuals",  lag.max = 40)
pacf(resid_best, main = "PACF of Residuals", lag.max = 40)

# periodogram
spec.pgram(resid_best, taper = 0, log = "no",
           main = "Periodogram of Residuals (Spectral Analysis)")
abline(h = var(resid_best), col = "red", lty = 2)
legend("topright", legend = "Flat spectrum (white noise level)",
       col = "red", lty = 2, cex = 0.8)

# Histogram of residuals
hist(resid_best, breaks = 50, probability = TRUE,
     main = "Histogram of Residuals", xlab = "Residuals", col = "lightblue")
curve(dnorm(x, mean = mean(resid_best), sd = sd(resid_best)),
      add = TRUE, col = "red", lwd = 2)
par(mfrow = c(1,1))

# white noise tests
cat("\nLjung-Box test on residuals (lag 10):\n")
print(Box.test(resid_best, lag = 10, type = "Ljung-Box", fitdf =
sum(!is.na(coef(best_model)))))

cat("\nLjung-Box test on residuals (lag 20):\n")
print(Box.test(resid_best, lag = 20, type = "Ljung-Box", fitdf =
sum(!is.na(coef(best_model)))))
```

```r

cat("\nShapiro-Wilk normality test (on 5000 random residuals):\n")
set.seed(42)
sw_idx <- sample(length(resid_best), min(5000, length(resid_best)))
print(shapiro.test(resid_best[sw_idx]))


#  model parameters
cf <- coef(best_model)
se <- sqrt(diag(vcov(best_model)))

param_table <- data.frame(
  Parameter = names(cf),
  Estimate  = round(cf, 6),
  Std_Error = round(se, 6),
  t_value   = round(cf / se, 3),
  Signif    = ifelse(abs(cf/se) > 2.576, "***",
                     ifelse(abs(cf/se) > 1.960, "**",
                            ifelse(abs(cf/se) > 1.645, "*", "")))
)
rownames(param_table) <- NULL
print(param_table)


#
#  9: Predictions using hypothetical inputs
# Scenario matrix: what if Nasdaq has various hypothetical 1-day log returns?
hyp_nasdaq <- c(-0.04, -0.02, -0.01, 0.00, 0.01, 0.02, 0.04)
labels      <- c("-4%", "-2%", "-1%", "0%", "+1%", "+2%", "+4%")

# Use the last state of the model to forecast 1- ahead for each scenario
pred_sp500 <- numeric(length(hyp_nasdaq))
for (i in seq_along(hyp_nasdaq)) {
  fc <- forecast(best_model, h = 1,
                 xreg = matrix(hyp_nasdaq[i], nrow = 1))
  pred_sp500[i] <- as.numeric(fc$mean)
}

pred_table <- data.frame(
  Nasdaq_Input   = labels,
  Nasdaq_LogRet  = round(hyp_nasdaq, 4),
  SP500_Predicted = round(pred_sp500, 6),
  SP500_Pct       = paste0(round(pred_sp500 * 100, 3), "%")
)
cat("One--ahead S&P 500 log return predictions:\n")
print(pred_table)

# Plot predictions
par(mar = c(5,5,3,2))
plot(hyp_nasdaq * 100, pred_sp500 * 100,
     type = "b", pch = 16, col = "blue", lwd = 2,
     xlab = "Hypothetical Nasdaq Log Return (%)",
     ylab = "Predicted S&P 500 Log Return (%)",
     main = "ARMAX: Predicted S&P 500 Return vs Hypothetical Nasdaq Input")
```

```r
abline(h = 0, lty = 2, col = "grey")
abline(v = 0, lty = 2, col = "grey")
text(hyp_nasdaq * 100, pred_sp500 * 100,
     labels = pred_table$SP500_Pct, pos = 4, cex = 0.8, col = "red")

```